MLP Level 1 Viva

Top 25 Most Asked Questions & Model Answers

Compiled from student responses across multiple exam batches

■ **How to Use This Guide** | Questions are ranked by frequency from student reports. ★★★★★ = asked in nearly every viva. Read the answers aloud before your viva — especially Q1 (notebook walkthrough) and Q4 (pipeline coding). Know the reasoning behind every choice in your notebook.

1 1. Walk me through your notebook / Explain what you have done.

★★★★★ (Asked in virtually every viva)

- Start with the **Problem Statement** — what are we predicting and why does it matter?
- Move to **EDA** — describe key visualisations, distributions, and 2–3 concrete insights (e.g., class imbalance, skewed target, correlated features).
- Explain **Preprocessing** — missing-value handling, encoding, scaling — with the reasoning behind each choice.
- Cover **Feature Engineering** — new columns created, features dropped, and why.
- Describe **Model Training** — which models you tried, why you chose them, and how you compared them.
- Conclude with **Hyperparameter Tuning** and your final best model with its score.

■ *Tip: Aim for 15–20 minutes. Speak continuously so the proctor has fewer opportunities to ask extra questions.*

2 2. What is Hyperparameter Tuning? Why is it needed?

★★★★★

- **Hyperparameters** are configuration settings chosen *before* training (e.g., n_estimators, max_depth, learning_rate). They are not learned from data.
- **Why tune?** The default values are rarely optimal for your specific dataset. Tuning finds the combination that maximises model performance.
- **GridSearchCV** tries every combination in an exhaustive grid. It is thorough but slow for large grids.
- **RandomizedSearchCV** samples a fixed number of combinations (n_iter) randomly, making it much faster with comparable results for large search spaces.
- **Cross-validation (cv)** within each method ensures the score is not dependent on a single train/validation split.

■ *Tip: Know how many total fits happen — GridSearchCV: len(param_grid) × cv folds; RandomizedSearchCV: n_iter × cv folds.*

3 3. What is GridSearchCV vs RandomizedSearchCV? When to use which?

★★★★★

- **GridSearchCV** exhaustively evaluates all combinations. Best when the search space is small and you want the guaranteed optimal combination.
- **RandomizedSearchCV** evaluates a random subset (n_iter combinations). Best when the search space is large — it is faster and often finds near-optimal results.
- Both use cross-validation internally to evaluate each parameter set.
- Example: If you have 3 hyperparameters with 4 values each, GridSearch runs 64 × cv fits; RandomizedSearch can be limited to, say, 20 × cv fits.

■ *Tip: Mentioning n_iter and cv parameters shows depth of understanding.*

4 4. What is a Pipeline? Why use it? ★★★★★

- A **Pipeline** chains a sequence of transformers and an estimator into a single object.
- **Key benefit**: Prevents *data leakage* — fit is called only on training data and transform is applied consistently to test data.
- Makes code cleaner, reproducible, and easy to tune with GridSearchCV.
- Example: `Pipeline([('imputer', SimpleImputer()), ('scaler', StandardScaler()), ('model', LogisticRegression())])`
- **ColumnTransformer** is used to apply different pipelines to numerical and categorical columns simultaneously.

■ *Tip: Be ready to code a basic pipeline from scratch — it is the most common coding question at Level 1.*

5 5. What is the difference between Bagging and Boosting? ★★★★★

- **Bagging (Bootstrap Aggregating)**: Trains multiple models independently and in parallel on random subsets of data. Combines predictions by majority vote (classification) or averaging (regression). Reduces *variance*. Example: Random Forest.
- **Boosting**: Trains models sequentially; each new model focuses on the errors of the previous one. Combines predictions with weighted voting. Reduces *bias*. Example: XGBoost, LightGBM, GradientBoosting.
- Bagging is more robust to overfitting; Boosting achieves higher accuracy but can overfit if not regularised.

■ *Tip: 'Weak learners' in boosting are models slightly better than random guessing (e.g., shallow decision trees).*

6 6. What is Overfitting and Underfitting? How do you detect and fix them? ★★★★★

- **Overfitting**: Model memorises training data — high train score, low test score (high variance). Fix: reduce model complexity, add regularisation, use more data, pruning, dropout.
- **Underfitting**: Model is too simple to capture patterns — low train and test score (high bias). Fix: increase model complexity, add features, reduce regularisation, train longer.

- **Detection**: Compare train vs validation score. Large gap → overfitting; both scores low → underfitting.
- **Bias-Variance Tradeoff**: Simpler models have high bias (underfit); complex models have high variance (overfit). Goal is to find the sweet spot.

■ *Tip: For tree-based models, reducing max_depth and n_estimators helps with overfitting.*

7 7. What is One-Hot Encoding (OHE)? How is it different from Label Encoding? ★★★★★

- **Label Encoding**: Assigns an integer to each category (e.g., Red→0, Green→1, Blue→2). Implies an ordinal relationship that may not exist. Best for ordinal features or tree-based models.
- **One-Hot Encoding**: Creates a new binary column for each category. No ordinal assumption. Best for nominal features and linear/distance-based models.
- **handle_unknown='ignore'**: Silently ignores unseen categories during transform (useful for test data).
- **drop='first'**: Drops one column per feature to avoid multicollinearity (dummy variable trap).
- **sparse_output=False**: Returns a dense array instead of a sparse matrix — needed when passing output to pandas.

■ *Tip: Label Encoding can mislead distance-based algorithms (KNN, SVM) by implying numeric order.*

8 8. What is Feature Scaling? Why is it needed? What are the types? ★★★★★

- Scaling brings all numerical features to a comparable range so distance-based and gradient-descent algorithms are not dominated by large-magnitude features.
- **StandardScaler**: Subtracts mean, divides by std. Output has mean=0, std=1. Assumes roughly normal distribution.
- **MinMaxScaler**: Scales to [0, 1]. Sensitive to outliers.
- **RobustScaler**: Uses median and IQR — robust to outliers. Best when data has significant outliers.
- Tree-based models (Random Forest, XGBoost) do *not* require scaling; linear models, KNN, SVM, and neural networks do.

■ *Tip: Always fit the scaler on training data only, then transform both train and test — this is automatic inside a Pipeline.*

9 9. What is SimpleImputer? Explain its strategies. ★★★★★

- **SimpleImputer** fills missing values in a dataset.
- strategy='mean' — fill with column mean. Best for normally distributed numerical features.
- strategy='median' — fill with column median. Best for skewed numerical features or when outliers are present.
- strategy='most_frequent' — fill with mode. Used for categorical or binary features.

- strategy='constant' + fill_value — fill with a specified constant.
- Always call `.fit()` on training data, `.transform()` on both train and test.

■ *Tip: A frequent coding task — be ready to write SimpleImputer inside a ColumnTransformer pipeline.*

10 10. Explain Precision, Recall, F1-Score and Accuracy. When to use which? ★★★★★

- **Accuracy** = (TP + TN) / Total. Misleading when classes are imbalanced.
- **Precision** = TP / (TP + FP). Of all positives predicted, how many are actually positive? Use when false positives are costly (spam detection).
- **Recall (Sensitivity)** = TP / (TP + FN). Of all actual positives, how many did we catch? Use when false negatives are costly (disease diagnosis).
- **F1-Score** = 2 × (Precision × Recall) / (Precision + Recall). Harmonic mean — balances both. Best metric for imbalanced datasets.
- **Support** in classification report = actual number of samples of that class.
- **Macro avg**: unweighted mean across classes. **Weighted avg**: weighted by support.

■ *Tip: For poisonous-apple / disease detection → maximise Recall. For spam filter → maximise Precision.*

11 11. Explain the Confusion Matrix. Identify TP, TN, FP, FN. ★★★★★

- A confusion matrix is a 2×2 table (for binary classification) showing actual vs predicted classes.
- **TP (True Positive)**: Predicted Positive, Actual Positive.
- **TN (True Negative)**: Predicted Negative, Actual Negative.
- **FP (Type I Error)**: Predicted Positive, Actual Negative.
- **FN (Type II Error)**: Predicted Negative, Actual Positive.
- Row = Actual class; Column = Predicted class (sklearn convention).

■ *Tip: Diagonal elements are correct predictions; off-diagonal are errors.*

12 12. What is Cross-Validation (CV)? How does it work? ★★★★★

- Cross-validation evaluates a model on multiple train/validation splits to get a more reliable estimate of generalisation performance.
- **k-Fold CV**: Data is split into k folds. The model trains on k-1 folds and validates on the remaining 1 fold, repeated k times. Final score = average over k runs.
- Prevents the evaluation from depending on a single lucky/unlucky split.
- cv=5 in sklearn means 5-fold cross-validation.
- **Stratified k-Fold**: Preserves class proportions in each fold — important for imbalanced datasets.

■ *Tip: Number of model fits with GridSearchCV = len(param_combinations) × cv.*

13 13. How does Logistic Regression work? ★★★★■

- Despite the name, Logistic Regression is a *classification* algorithm.
- It models the probability of a class using the **sigmoid function**: $\sigma(z) = 1 / (1 + e^{-z})$, where $z = w^T x + b$.
- Output is a probability [0, 1]. A threshold (default 0.5) converts it to a class label.
- **Loss function**: Binary Cross-Entropy (Log Loss) = –[y·log(■) + (1–y)·log(1–■)].
- **Key hyperparameters**: C (inverse regularisation strength — smaller C = more regularisation), penalty ('l1', 'l2', 'elasticnet'), solver.

■ *Tip: Logistic Regression is a parametric, discriminative model. It is called 'Regression' because it models a continuous probability.*

14 14. What is Random Forest? Is it Bagging or Boosting? ★★★★■

- Random Forest is a **Bagging** algorithm — it trains many decision trees independently on bootstrap (random with replacement) samples of the data.
- Each tree also uses a random subset of features at each split (feature randomisation), which reduces correlation between trees.
- Final prediction = majority vote (classification) or average (regression).
- It is a **non-parametric** model — there are no closed-form parameters; the model is the collection of trees.
- **Key hyperparameters**: n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features.

■ *Tip: Random Forest reduces variance compared to a single decision tree. More trees generally improve stability but increase training time.*

15 15. What is XGBoost? How does it differ from Random Forest? ★★★★■

- XGBoost (Extreme Gradient Boosting) is a **Boosting** algorithm — trees are built sequentially, each correcting the residual errors of the previous one.
- Uses **gradient descent** to minimise the loss function, adding trees that point in the direction of steepest error reduction.
- Includes built-in **regularisation** (L1/L2) to prevent overfitting.
- **Key hyperparameters**: n_estimators (number of trees), learning_rate (shrinkage), max_depth, subsample, colsample_bytree.
- If model overfits: decrease n_estimators, decrease learning_rate, decrease max_depth.
- Random Forest: parallel trees, reduces variance. XGBoost: sequential trees, reduces bias.

■ *Tip: LightGBM is similar to XGBoost but uses leaf-wise tree growth making it faster on large datasets.*

16 16. What is Exploratory Data Analysis (EDA)? What do you look for? ★★★★■

- EDA is the process of visually and statistically exploring a dataset to understand its structure, patterns, and anomalies before modelling.
- **What to examine**: Shape of dataset, data types, missing values, target variable distribution, feature distributions (histograms, boxplots), correlations (heatmap), outliers, class balance.
- **Key plots**: Histogram (distribution), Boxplot (outliers/spread), Heatmap (correlations), Countplot (class balance), Scatter plot (bivariate relationship).
- **Insights to articulate**: Which features are correlated with the target? Are there outliers? Is the data imbalanced?

■ *Tip: Add markdown cells with at least 2–3 written insights after each EDA section — examiners love this.*

17 17. What is Data Preprocessing? What are its steps? ★★★★■

- Data Preprocessing transforms raw data into a clean, model-ready format.
- **Steps**: (1) Handle missing values (imputation), (2) Handle outliers, (3) Encode categorical variables, (4) Feature scaling, (5) Feature selection / dimensionality reduction, (6) Train-test split.
- It is essential because most ML algorithms cannot handle missing values, require numeric input, and are sensitive to feature scale.

■ *Tip: Always perform preprocessing after the train-test split to prevent data leakage.*

18 18. What is Feature Engineering? Give examples. ★★★★■

- Feature Engineering creates new informative features from existing ones to improve model performance.
- **Examples**: Extracting day, month, year from a datetime column; creating interaction terms (feature_A × feature_B); log-transforming a skewed feature; calculating ratios (price_per_sqft = price / area).
- It leverages domain knowledge to encode relationships the model may not discover automatically.
- Good feature engineering often improves performance more than hyperparameter tuning.

■ *Tip: Mention 1–2 concrete feature engineering steps you applied in your notebook and explain why each helped.*

19 19. What is Dimensionality Reduction? How does PCA work? ★★★■■

- **Dimensionality Reduction** reduces the number of features while preserving as much information as possible. It combats the curse of dimensionality, reduces training time, and can mitigate overfitting.
- **PCA (Principal Component Analysis)**: An unsupervised, linear technique that projects data onto directions (principal components) of maximum variance.
- Steps: (1) Standardise the data, (2) Compute the covariance matrix, (3) Compute eigenvectors and eigenvalues, (4) Select top-k eigenvectors as principal components, (5) Project data.
- PCA does *not* eliminate data points — it reduces features. It does not discard rows.
- PCA is unsupervised and linear. Kernel PCA handles non-linear data.

■ *Tip: Use explained_variance_ratio_ to decide how many components to keep (e.g., 95% variance explained).*

20 20. What are Parametric vs Non-Parametric models? Give examples. ★★★■■

- **Parametric models**: Assume a specific functional form; have a fixed, finite number of parameters learned from data. Fast to train and predict. E.g., Logistic Regression, Linear Regression, Naive Bayes, Perceptron.

- **Non-parametric models**: Make fewer assumptions; the number of parameters can grow with data. More flexible but computationally heavier. E.g., Decision Trees, Random Forest, KNN, SVM (with kernel).
- Random Forest is non-parametric — the model IS the collection of trees, with no fixed parameter count.

■ *Tip: The 'parameters' in parametric models are weights/coefficients. Hyperparameters (like n_estimators) are different from model parameters.*

21 21. Why is random_state=42 used? ★★★■■

- **random_state** seeds the random number generator, ensuring that any randomness (e.g., train-test split, tree construction) produces the same result every time the code is run.
- This makes experiments **reproducible** — another person running the same code gets identical results.
- The value 42 is a convention with no mathematical significance (popularised by The Hitchhiker's Guide to the Galaxy). Any integer would work equally well.

■ *Tip: Some examiners are tired of 'Hitchhiker's Guide' — just say it's a seed for reproducibility and any value works.*

22 22. What is KNN? Why is it called a 'lazy learner'? ★★★■■

- **K-Nearest Neighbours (KNN)**: For a query point, finds the k closest training points (using Euclidean or other distance) and predicts by majority vote (classification) or average (regression).
- **Lazy learner**: KNN has no training phase — it simply memorises the entire training set. All computation happens at prediction time.
- Pros: Simple, no assumptions about data distribution. Cons: Slow at inference for large datasets, sensitive to feature scale (requires normalisation), and to irrelevant features.
- Choosing k: small k → overfitting; large k → underfitting.

■ *Tip: KNN requires scaling because it relies on distances — unscaled features with large ranges dominate the distance calculation.*

23 23. What is train_test_split? Why do we need it? ★★★■■

- **train_test_split** divides the dataset into a training set (for model fitting) and a test set (for unbiased evaluation).
- Without this split, evaluating the model on training data gives an overly optimistic score (the model has 'seen' that data).
- **stratify=y** ensures class proportions are preserved in both splits — critical for imbalanced datasets.
- Typical splits: 80/20 or 70/30. test_size=0.2 means 20% goes to test.

■ *Tip: Common coding task — be ready to write: from sklearn.model_selection import train_test_split; X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)*

24 24. What is R^2 Score (Coefficient of Determination)? ★★★■■

- R^2 measures how much of the variance in the target variable is explained by the model.
- Formula: $R^2 = 1 - SS_res / SS_tot$, where $SS_res = \Sigma(y - ■)^2$ (residuals) and $SS_tot = \Sigma(y - ■)^2$ (total variance).
- Range: $-\infty$ to 1. $R^2 = 1 \rightarrow$ perfect fit; $R^2 = 0 \rightarrow$ model no better than predicting mean; $R^2 < 0 \rightarrow$ model worse than mean.
- R^2 can be misleading for non-linear models; RMSE gives error in the original units, which is more interpretable.

■ *Tip: For regression problems, also mention RMSE = √(mean of squared residuals) as a complementary metric.*

25 25. What is Handling Class Imbalance? Techniques? ★★★■■

- Class imbalance occurs when one class is significantly more frequent than another (e.g., 95% negative, 5% positive). Accuracy becomes a misleading metric.
- **Oversampling**: Generate synthetic minority-class samples. SMOTE creates interpolated samples between minority-class neighbours.
- **Undersampling**: Remove majority-class samples (loses data). Clustering-based undersampling creates 200 clusters of majority class and selects one point per cluster.
- **Class weights**: Pass class_weight='balanced' to the model — penalises misclassification of minority class more.
- **Better metric**: Use F1-score, Precision-Recall AUC, or ROC-AUC instead of accuracy.

■ *Tip: Apply SMOTE/oversampling only on training data to avoid leaking information about the test set distribution.*

■ Universal Level 1 Viva Tips from Students

① Explain your notebook for 15–20 min without pausing — it leaves less time for questions.
② Know every line of your code and *why* you wrote it, not just what it does.
③ Always have EDA insights written as markdown cells in your notebook.
④ Practice coding pipelines, SimpleImputer, and loading sklearn datasets (Iris, breast_cancer) in Colab.
⑤ If you do not know something, say so honestly and move on — examiners respect honesty.
⑥ For Level 2: Prepare MLT theory thoroughly, expect live coding, and know model mathematics.